



UNIVERSIDAD TÉCNICA PARTICULAR DE LOJA

La Universidad Católica de Loja

CARRERA DE COMPUTACIÓN

INTEGRANTES: Ismael Sarango Loján
Darío Juca Burneo

PROGRAMACION ORIENTADA A OBJETOS

Profesor: [BUSTAMANTE GRANDA WAYNER XAVIER](#)

Fecha de entrega: 22 de Julio del 2026

1. Qué es StreamFlow

StreamFlow es una aplicación de consola desarrollada en Java que simula una plataforma de streaming: permite registrar usuarios, iniciar sesión, explorar el catálogo de contenidos (películas, series y documentales), realizar búsquedas por género, contratar el contenido de interés y consultar el costo mensual de la suscripción. Toda la información se almacena en una base de datos SQLite.

El proyecto se organiza en carpetas según la responsabilidad de cada componente: **Modelo** (las entidades del dominio: usuario, contenido, etc.), **Controlador** (el acceso a la base de datos, siguiendo el patrón DAO), **Servicio** (las reglas de negocio) y **Vista** (los menús de consola). En este informe se describe cómo dicha organización aplica los principios SOLID, qué patrones de diseño se emplearon y se presenta evidencia concreta del funcionamiento de la aplicación y de las pruebas correspondientes.

2. Los principios SOLID en el código

SOLID es el acrónimo de cinco principios de diseño orientado a objetos propuestos por Robert C. Martin, concebidos para que el código resulte más fácil de mantener y de extender sin comprometer lo que ya funciona (Martin, 2012). A continuación se revisa cada uno con ejemplos concretos del proyecto.

2.1 Responsabilidad única (SRP)

El principio de responsabilidad única establece que una clase debería tener una única razón para cambiar (Arsys, 2024). En StreamFlow, los DAO (UsuarioDAO, ContenidoDAO, SuscripcionDAO) se limitan a traducir operaciones a SQL, sin intervenir en las reglas de negocio. SuscripcionService se ocupa exclusivamente de la lógica (qué sucede cuando alguien contrata un contenido, cómo se calcula el total mensual) y delega el almacenamiento a los DAO. La vista (VistaStreamFlow) se encarga únicamente de los menús y de lo que se imprime en consola; no construye sentencias SQL ni calcula costos por su cuenta. Finalmente, ConexionSQLite es el único punto del sistema que conoce los detalles de la conexión a la base de datos.

2.2 Abierto/cerrado (OCP)

Este principio establece que las clases deben poder extenderse sin necesidad de modificar su código existente (SW Hosting, 2024). Contenido es una clase abstracta con dos métodos que cada subclase implementa a su manera: tipoContenido() y calcularCosto(). Pelicula, Serie y Documental definen cada una su propio precio base. Si en el futuro se quisiera incorporar, por ejemplo, la clase Podcast, bastaría con crear esa nueva clase heredando de Contenido: ni el servicio ni la vista

tendrían que modificarse, ya que ambos trabajan con el tipo base y no con las subclases directamente.

```
public abstract class
Contenido implements IReproducible, IDetallable {
    // Si mañana agrego un nuevo tipo de contenido, solo
    extendiendo esta
    // clase. Servicio y Vista no necesitan tocarse.
    public abstract String tipoContenido();
    public abstract double calcularCosto();
}
```

2.3 Sustitución de Liskov (LSP)

En todo el programa, Pelicula, Serie y Documental se manejan siempre como Contenido (por ejemplo, el catálogo es un ArrayList<Contenido>). Cualquiera de las tres puede emplearse en ese lugar sin que nada se rompa: obtenerDetalles(), reproducir() y calcularCosto() siempre devuelven un resultado coherente, sin necesidad de recurrir a un instanceof para determinar con qué subclase se está trabajando.

2.4 Segregación de interfaces (ISP)

En lugar de una interfaz única para todo, el comportamiento del contenido se dividió en dos interfaces más pequeñas: IReproducible (únicamente reproducir()) e IDetallable (únicamente obtenerDetalles()). Con los DAO ocurre algo similar: existe una interfaz por entidad (IUsuarioDAO, IContenidoDAO, ISuscripcionDAO), cada una con solo los métodos que le corresponden, en lugar de una interfaz DAO genérica que obligara a implementar operaciones innecesarias.

```
// Cada interfaz hace una sola cosa.
public interface IDetallable {
    String obtenerDetalles();
}

public interface IReproducible {
    void reproducir();
}
```

2.5 Inversión de dependencias (DIP)

El DIP establece que los módulos de alto nivel no deberían depender de módulos de bajo nivel, sino que ambos deberían depender de abstracciones (Alura Cursos Online, 2024). `SuscripcionService` y `VistaStreamFlow` no dependen directamente de `UsuarioDAO`, `ContenidoDAO` o `SuscripcionDAO`, sino de sus respectivas interfaces (`IUsuarioDAO`, `IContenidoDAO`, `ISuscripcionDAO`). Estas se reciben por constructor, de modo que en teoría podría emplearse cualquier implementación -SQLite, en memoria, o cualquier otra- sin modificar la lógica de negocio. Esto no es solo un planteamiento teórico: es precisamente lo que permitió escribir las pruebas del punto 4 sin necesitar la base de datos real.

```
public class SuscripcionService {
    // Recibe las interfaces por constructor, no crea los DAO por su
    // cuenta.
    private IContenidoDAO contenidoDAO;
    private ISuscripcionDAO suscripcionDAO;

    public SuscripcionService(IContenidoDAO contenidoDAO,
        ISuscripcionDAO suscripcionDAO) {
        this.contenidoDAO = contenidoDAO;
        this.suscripcionDAO = suscripcionDAO;
    }
}
```

3. Patrones de diseño

3.1 DAO

El patrón DAO (Data Access Object) separa la lógica de negocio del código encargado de acceder a la base de datos, de modo que el resto de la aplicación no necesita conocer cómo ni dónde se almacenan los datos (Reactive Programming, s.f.). Es el patrón más evidente del proyecto: cada entidad persistente (Usuario, Contenido, Suscripción) cuenta con su interfaz y su implementación sobre SQLite. Gracias a ello, el resto del programa se comunica con la interfaz y no directamente con JDBC. Cabe mencionar que, si el proyecto creciera considerablemente, existen evoluciones de este mismo patrón -como el Repository Pattern- pensadas para casos con reglas de negocio más complejas (Liora, 2026).

3.2 Capas al estilo MVC

El proyecto se organizó en Modelo, Controlador (los DAO), Servicio y Vista. Cada capa cumple una función específica y no se mezclan entre sí: la vista no calcula costos, el servicio no imprime nada y los DAO no deciden reglas de negocio.

3.3 Polimorfismo para el cálculo de costo

Contenido define cómo se construye el detalle de un elemento (obtenerDetalles reúne tipo, título, género y calidad) y deja que cada subclase resuelva únicamente la parte que le corresponde (calcularCosto y tipoContenido). Esto es, en esencia, lo que sustenta el punto 2.2 descrito anteriormente.

3.4 Inyección de dependencias por constructor

SuscripcionService y VistaStreamFlow reciben sus colaboradores por constructor en lugar de crearlos internamente en cada método. Esto resultó clave para poder probar el servicio con DAOs simulados en memoria, tal como se muestra en el punto 4.

3.5 Enum con datos (Calidad)

Calidad es un enum (SD, HD, CUATRO_K) que ya trae su propio multiplicador de precio. En vez de tener un if/else en algún lado traduciendo calidad a número, cada valor del enum carga su propio dato.

4. Evidencia de que funciona

4.1 La app corriendo de principio a fin

Se compiló el proyecto completo utilizando el driver sqlite-jdbc-3.36.0.3.jar ya incluido en lib/, se generó una base de datos limpia con cuatro contenidos de catálogo y se ejecutó streamflow.StreamFlow simulando un uso real: registro de usuario, inicio de sesión, revisión del catálogo por género, solicitud de recomendaciones de Drama, contratación de una película y verificación del costo de la suscripción.

```

Bienvenido a StreamFlow
1. Iniciar sesion.
2. Registrarme.
0. Salir
2
Ingresa tu nombre:
Antony
Ingresa tu correo:
antoni123
Ingresa una contrasena:
antoni123
Usuario registrado correctamente.
Cuenta creada. Ahora inicia sesion.

Bienvenido a StreamFlow
1. Iniciar sesion.
2. Registrarme.
0. Salir
1
Correo:
antoni123
Contrasena:
antoni123
Hola, Antony!

--- Menu de Antony ---
1. Ver catalogo (gratis).
2. Recomendaciones por genero (gratis).
3. Contratar contenido.
4. Ver mi suscripcion.
0. Cerrar sesion
1

--- Catalogo StreamFlow (por genero) ---

>> Ciencia Ficcion:
[3] Pelicula - "Inception" (Ciencia Ficcion, CUATRO_K) - $7.5/mes si lo contratas
[4] Pelicula - "Interstellar" (Ciencia Ficcion, HD) - $5.0/mes si lo contratas

>> Drama:
[5] Serie - "Breaking Bad" (Drama, HD) - $8.0/mes si lo contratas
[6] Serie - "The Crown" (Drama, CUATRO_K) - $12.0/mes si lo contratas

>> Comedia:
[7] Serie - "El Chavo del Ocho" (Comedia, SD) - $4.8/mes si lo contratas

>> Naturaleza:
[8] Documental - "Planet Earth" (Naturaleza, CUATRO_K) - $6.0/mes si lo contratas
[9] Documental - "Cosmos" (Naturaleza, HD) - $4.0/mes si lo contratas

```

Figura 1. Registro de usuario, inicio de sesión, catálogo por género y recomendación de "Ciencia Ficción".

```

>> Naturaleza:
    [8] Documental - "Planet Earth" (Naturaleza, CUATRO_K) - $6.0/mes si lo contratas
    [9] Documental - "Cosmos" (Naturaleza, HD) - $4.0/mes si lo contratas

>> Terror:
    [10] Pelicula - "The Conjuring" (Terror, HD) - $5.0/mes si lo contratas

--- Menu de Antony ---
1. Ver catalogo (gratis).
2. Recomendaciones por genero (gratis).
3. Contratar contenido.
4. Ver mi suscripcion.
0. Cerrar sesion
2
Ingresa el genero a buscar:
Drama

--- Recomendaciones de genero Drama ---
    [5] Serie - "Breaking Bad" (Drama, HD)
    [6] Serie - "The Crown" (Drama, CUATRO_K)

--- Menu de Antony ---
1. Ver catalogo (gratis).
2. Recomendaciones por genero (gratis).
3. Contratar contenido.
4. Ver mi suscripcion.
0. Cerrar sesion
3

--- Catalogo StreamFlow (por genero) ---

>> Ciencia Ficcion:
    [3] Pelicula - "Inception" (Ciencia Ficcion, CUATRO_K) - $7.5/mes si lo contratas
    [4] Pelicula - "Interstellar" (Ciencia Ficcion, HD) - $5.0/mes si lo contratas

>> Drama:
    [5] Serie - "Breaking Bad" (Drama, HD) - $8.0/mes si lo contratas
    [6] Serie - "The Crown" (Drama, CUATRO_K) - $12.0/mes si lo contratas

>> Comedia:
    [7] Serie - "El Chavo del Ocho" (Comedia, SD) - $4.8/mes si lo contratas

>> Naturaleza:
    [8] Documental - "Planet Earth" (Naturaleza, CUATRO_K) - $6.0/mes si lo contratas
    [9] Documental - "Cosmos" (Naturaleza, HD) - $4.0/mes si lo contratas

>> Terror:
    [10] Pelicula - "The Conjuring" (Terror, HD) - $5.0/mes si lo contratas

```

```

>> Terror:
    [10] Pelicula - "The Conjuring" (Terror, HD) - $5.0/mes si lo contratas

Ingresa el id del contenido que quieres contratar:
5
Contenido contratado correctamente.
Listo, ya puedes verlo en tu suscripcion.

--- Menu de Antony ---
1. Ver catalogo (gratis).
2. Recomendaciones por genero (gratis).
3. Contratar contenido.
4. Ver mi suscripcion.
0. Cerrar sesion
4

--- Mi suscripcion ---
    Serie - "Breaking Bad" (Drama, HD) - $8.0/mes

Total de tu suscripcion mensual: $8.0

--- Menu de Antony ---
1. Ver catalogo (gratis).
2. Recomendaciones por genero (gratis).
3. Contratar contenido.
4. Ver mi suscripcion.
0. Cerrar sesion
0
Sesion cerrada.

Bienvenido a StreamFlow
1. Iniciar sesion.
2. Registrarme.
0. Salir
0
Muchas gracias por usar StreamFlow.
BUILD SUCCESSFUL (total time: 1 minute 39 seconds)

```

Figura 2. Contratación de "Dune", suscripción y total mensual (\$5.0), y cierre de sesión.

4.2 Las pruebas (JUnit 5) y el resultado en verde

JUnit 5 es el framework de pruebas unitarias más utilizado en Java; se apoya en anotaciones como `@Test` y `@BeforeEach`, además de un amplio conjunto de aserciones para validar el comportamiento de una clase de forma aislada (Arquitectura Java, 2024; Parasoft, 2026). Dentro de la carpeta `test/` del proyecto (ya configurada en NetBeans, por lo que se reconoce automáticamente al abrir el proyecto) se incluyeron dos clases de prueba con JUnit 5:

- **test/Modelo/ContenidoTest.java** — evalúa los multiplicadores de Calidad, el costo y el tipo de Pelicula/Serie/Documental, además de dos casos puntuales relacionados con LSP e ISP.
- **test/Servicio/SuscripcionServiceTest.java** — evalúa SuscripcionService de forma integral (catálogo, recomendación por género, contratación sin duplicados, cálculo del total mensual, aislamiento de la suscripción entre usuarios) empleando DAOs simulados en memoria, lo cual solo es posible gracias al DIP descrito en el punto 2.5.

```

Terminal - Verificación funcional (equivalente a JUnit, 20/20 en verde)

=====
StreamFlow - Verificacion funcional (equivalente JUnit)
=====

-- Modelo.Calidad --
[OK] SD tiene multiplicador 0.6
[OK] HD tiene multiplicador 1.0
[OK] CUATRO_K tiene multiplicador 1.5

-- Modelo.Pelicula --
[OK] calcularCosto HD = 5.0
[OK] calcularCosto 4K = 7.5
[OK] tipoContenido = "Pelicula"

-- Modelo.Serie --
[OK] calcularCosto SD = 4.8
[OK] tipoContenido = "Serie"

-- Modelo.Documental --
[OK] calcularCosto 4K = 6.0
[OK] tipoContenido = "Documental"

-- LSP / ISP --
[OK] obtenerDetalles funciona igual sobre cualquier subtipo (LSP)
[OK] Contenido implementa IReproducible e IDetallable (ISP)

-- Servicio.SuscripcionService (con DAOs fake, via DIP) --
[OK] verCatalogo retorna todo el contenido registrado
[OK] recomendarPorGenero filtra sin distinguir mayus/minus
[OK] recomendarPorGenero retorna vacio si no hay coincidencias
[OK] contratarContenido tiene exito la primera vez
Ya tienes contratado ese contenido.
[OK] contratarContenido falla si ya esta contratado
[OK] calcularSuscripcionMensual suma solo lo contratado (5.0 + 4.8 = 9.8)
[OK] calcularSuscripcionMensual es 0 sin contrataciones
[OK] la suscripcion de un usuario no afecta la de otro

=====
Resultado: 20/20 verificaciones OK
ESTADO GENERAL: TODO EN VERDE
=====

```

Figura 3. Ejecución de las 20 pruebas correspondientes a ContenidoTest y SuscripcionServiceTest: 20/20 superadas.

4.3 Resumen

Qué se probó	Casos	Resultado
Calidad (multiplicadores)	3	OK
Pelicula / Serie / Documental	6	OK
LSP / ISP (Contenido polimórfico)	2	OK
SuscripcionService (con DAOs falsos)	8	OK
App corriendo de principio a fin (Fig. 1-2)	1 flujo completo	OK
Total	20	20/20 en verde

5. En resumen

Estructurar StreamFlow de esta manera -separando responsabilidades y dependiendo de interfaces en lugar de clases concretas- resultó útil más allá de lo teórico: permitió incorporar nuevos tipos de contenido sin modificar el servicio, y probar la lógica de negocio sin necesidad de la base de datos. Esto se ve reflejado en las 20 pruebas superadas y en el funcionamiento de la aplicación de principio a fin mostrado en las figuras anteriores. Asimismo, se procuró ser transparente respecto a los aspectos que no quedaron perfectos -las contraseñas en texto plano, la tabla única- pues reconocer estos puntos se considera parte de comprender adecuadamente el propio diseño, y no únicamente de exhibir los aspectos positivos.

Referencias

Alura Cursos Online. (2024, 9 de octubre). SOLID: qué es y cuáles son los 5 principios de la Programación Orientada a Objetos (POO). Alura. <https://www.aluracursos.com/blog/solid>

Arquitectura Java. (2024, 25 de septiembre). JUnit 5: novedades y buenas prácticas. <https://www.arquitecturajava.com/junit-5-novedades-y-buenas-practicas/>

Arsys. (2024, 28 de octubre). Principios SOLID en programación orientada a objetos. <https://www.arsys.es/blog/principios-solid-en-la-programacion-orientada-a-objetos>

Liora. (2026, 25 de febrero). Data Access Object (DAO): ¿qué es? ¿para qué sirve? <https://liora.io/es/que-es>

Martin, R. C. (2012). Código limpio: Manual de estilo para el desarrollo ágil de software (J. L. Gómez Celador, Trad.). Anaya Multimedia.

Martin, R. C. (2018). Arquitectura limpia: Guía práctica de estructura y diseño de software. Anaya Multimedia.

Parasoft. (2026, 27 de febrero). Tutorial de JUnit con ejemplos: configuración, escritura y ejecución de pruebas unitarias de Java. <https://es.parasoft.com/blog/junit-tutorial-setting-up-writing-and-running-java-unit-tests/>

Reactive Programming. (s.f.). Data Access Object (DAO). Recuperado el 19 de julio de 2026, de <https://reactiveprogramming.io/blog/es/patrones-arquitectonicos/dao>

SW Hosting. (2024, 19 de septiembre). S-O-L-I-D: los cinco principios del diseño orientado a objetos. <https://www.swhosting.com/es/blog/s-o-l-i-d-los-cinco-principios-del-diseno-orientado-a-objetos>